

Week 4 - Class 7: Operators and Conversions

Why This Matters

Operators allow arithmetic, comparison, logic, and assignment. Conversions control what happens with mixed types. Understanding precedence, division rules, and promotions is vital to avoid bugs.

Arithmetic Operators

Supported: + - * / %

In C++ int division truncates toward 0, not like Python's floor division.

```
#include <iostream>
using namespace std;
int main(){
    cout << 7/2 << "\n";    // 3
    cout << -7/2 << "\n";   // -3, truncates toward 0
}
```

Floor Division vs Truncation

In Python, // is floor division (rounds down): $-7//2 = -4$.

In C++, integer division truncates toward zero: $-7/2 = -3$.

Important difference.

Comparison and Logical Operators

== != < <= > >=

&& || !

Assignment and Compound

=, +=, -=, *=, /=, %= etc.

Operator Precedence

Simplified:

()

* / %

+ -

<< >>

< <= > >=

== !=

&&

||

= += -= ...

Implicit Conversions (Hierarchy)

Hierarchy (highest to lowest):

1. long double
2. double
3. float
4. unsigned long long
5. long long
6. unsigned long
7. long
8. unsigned int
9. int
10. char, bool, short

Examples: int+double -> double.

short in expressions -> int.

C++14 Fundamentals - Student Edition

Explicit Conversions

Use `static_cast<T>` for clarity.

Example: `double pi=3.14; int n=static_cast<int>(pi);`

Common Mistakes

1. Integer division surprises ($7/2=3$).
2. Signed/unsigned mix-ups ($-1<1u$).
3. Forgetting precedence.
4. Using C-style casts.

Exercises

1. Compute $7/2$ with `int` and `double`.
2. Test $-7/2$ in C++ vs Python $-7//2$.
3. Predict $3+4*5$ vs $(3+4)*5$.
4. Mix signed and unsigned.
5. Use `static_cast` to convert `double` to `int`.

Project Tie-In

Add to Type Explorer: show `int` vs `double` division, compare $-7/2$ vs Python $-7//2$, print mixed-type results showing promotions.